



Informe Taller 2

Manuel Jesús Rosero Zuñiga 202176007

1 INFORME DE PROCESOS

Esto es realizado tomando en cuenta lo explicado en el taller usaremos lo otorgado por “pertenece” y “mucho mayor que”, además de obviamente lo explicado por “ConjDifusos”

GRANDE

Se implemento la Función Grande que recibe dos números “d” el cual es un número mayor o igual a 1, luego recibe “e” el cual es un numero entero mayor a 1. Estos números son utilizados para calcular el grado de pertenencia de “d” en un conjunto difuso

En este caso que es para este se puede notar como haciendo la prueba con un conjunto entre 1 y 2, nos recibe dichos números en “d” y “e” respectivamente

```

  ▾ VARIABLES
    ▾ Local
      d = 1
      e = 2
      > this = ConjuntosDifusos@1618
    
```

```

  ▾ WATCH
    
```

Posteriormente la función recibe el valor al cual tratamos de encontrar su grado de pertenencia, en el caso de la prueba impuesta es 4 y por lo tanto su valor en “Int” es asignado a n. El programa para la siguiente parte ya teniendo todos los valores necesarios para la formula los aplica, o sea:

$$\left(\frac{n}{d + n}\right)^e$$

```
✓ VARIABLES
  ✓ Local
    d$1 = 1
    e$1 = 2
    n = 4

✓ WATCH
  n = 4
```

COMPLEMENTO

Se implemento la Función Complemento que recibe un “c” conjunto difuso para luego calcular cual es el complemento de un conjunto difuso, para eso debe de retornar otro conjunto difuso

En este caso lo primero que recibe la función es el conjunto “c” que se va a utilizar para sus respectivas operaciones, en este caso estamos usando un conjunto de mayoresQue entre 1 y 3

```
✓ VARIABLES 
  ✓ Local
    > c = ConjuntosDifusos$$Lambda$89/0x0000000800d02f68@1619 "ta...
    > this = ConjuntosDifusos@1620
```

Posteriormente recibe el valor de “n” el numero que vamos a evaluar si es un complemento del conjunto anteriormente mencionado

```
✓ VARIABLES 📄
  ✓ Local
    > $this = ConjuntosDifusos@1620
    > c$1 = ConjuntosDifusos$$Lambda$89/0x0000000800d02f68@1619 "...
      n = 2

  WATCH
    n = 2
```

Posteriormente se llama a la función “pertenece” asignándole a “s” el valor del conjunto anteriormente mencionado y a “elem” el valor que deseamos evaluar. Una vez tiene esos valores el programa simplemente tiene que hacer los cálculos básicos sin tener que llamar a más variables.

```
✓ Local
  elem = 2
  ✓ s = ConjuntosDifusos$$Lambda$89/0x0000000800d02f68@1619 "ta...
    arg$1 = 1
    arg$2 = 3
  > this = ConjuntosDifusos@1620
```

UNIÓN

Se implemento la Función Complemento que recibe dos conjuntos difusos para luego calcular cual es el grado de pertenencia de un elemento en la unión de ambos, para eso debe de retornar un valor de pertenencia

En este caso recibe dos conjuntos mayoresQue, uno entre 1 y 3 mientras el segundo es entre 2 y 5. En este caso vamos a evaluar es un “n” con valor de 4 como los anteriores

```
✓ VARIABLES
  ✓ Local
    > $this = ConjuntosDifusos@1958
    ✓ cd1$1 = ConjuntosDifusos$$Lambda$89/0x0000000800d02f68@1959...
      arg$1 = 1
      arg$2 = 3
    ✓ cd2$1 = ConjuntosDifusos$$Lambda$89/0x0000000800d02f68@1960...
      arg$1 = 2
      arg$2 = 5
      n = 4
```

Tal como en la anterior función procede a llamar a la función pertenece para asignarle a “s” el primer conjunto mencionado y a “elem” el valor de “n”

```
✓ VARIABLES
  ✓ Local
    elem = 4
    ✓ s = ConjuntosDifusos$$Lambda$89/0x0000000800d02f68@1959 "ta...
      arg$1 = 1
      arg$2 = 3
    > this = ConjuntosDifusos@1958
```

Finalmente lo mismo para poder evaluarlo con respecto al valor del segundo conjunto ahora siendo “s” entre 2 y 5, para luego tener los resultados de ambos “pertenece” antes de usar la operación “Max” para evaluar cual es el grado de pertenencia en la unión de ambos conjuntos

```

VARIABLES
  Local
    elem = 4
    s = ConjuntosDifusos$$Lambda$89/0x0000000800d02f68@1960 "ta..
    arg$1 = 2
    arg$2 = 5
    > this = ConjuntosDifusos@1958

```

INTERSECCIÓN

Se implemento la Función Intersección que recibe dos conjuntos difusos para luego calcular cual es el grado de pertenencia de un elemento en la unión de ambos, para eso debe de retornar otro un valor de pertenencia.

En este caso se vuelven a usar las mismas variables que antes en la que “cd1” es un conjunto entre 1 y 3, “cd2” es un conjunto entre 2 y 5, mientras que “n” tendrá un valor de 4. Entonces lo primero que la función va a recibir va a recibir dichos valores.

```

  ▾ Local
    > $this = ConjuntosDifusos@1958
  ▾ cd1$2 = ConjuntosDifusos$$Lambda$89/0x0000000800d06318@1959...
    arg$1 = 1
    arg$2 = 3
  ▾ cd2$2 = ConjuntosDifusos$$Lambda$89/0x0000000800d06318@1960...
    arg$1 = 2
    arg$2 = 5
    n = 4

```

Tal como antes lo que hace es llamar a la función de “pertenece” por lo que nuevamente empieza a asignar el valor de “n” a “elem” y el valor de los conjuntos a “s” empezando con “cd1”

```

  ▾ VARIABLES
    ▾ Local
      elem = 4
  ▾ s = ConjuntosDifusos$$Lambda$89/0x0000000800d06318@1959 "t...
    arg$1 = 1
    arg$2 = 3
  > this = ConjuntosDifusos@1958

```

Posteriormente hace lo mismo con “n” y “cd2” para tener los valores de pertenencia de “n” en ambos conjuntos, esto para aplicar la función “min” a ambos valores para que evalúen y tome el valor menor. Pues esto representa si esta en la intersección o no.

```
✓ VARIABLES
  ✓ Local
    elem = 4
    ✓ s = ConjuntosDifusos$$Lambda$89/0x0000000800d06318@1960 "ta...
      arg$1 = 2
      arg$2 = 5
    > this = ConjuntosDifusos@1958
```

INCLUSIÓN

Se implemento la Función Inclusión que recibe dos conjuntos difusos para luego verificar si un conjunto difuso puede pertenecer dentro de otro conjunto difuso, por lo que para eso usaremos recursión.

Con motivo del funcionamiento de la operación en la que debe de evaluar entre 0 y 1000, además del uso de recursión con cola esta operación es demasiado extensa por lo que solo se reducirá a una muestra entre 0 y 10 para agilizar y demostrar su funcionamiento. En este caso estamos probando con dos intervalos un “S2” entre 2 y 5, mientras el otro “S1” está entre 3 y 5. El resultado debe de ser positivo como se demostrará.

Como podemos ver con el programa este lo primero que recibe son los conjuntos asignados y son los que usará para lo posterior, pero primero llama la función auxiliar usando el intervalo entre 0 y 10 que mencionamos antes

```
✓ VARIABLES
  ✓ Local
    ✓ S1 = ConjuntosDifusos$$Lambda$89/0x0000000800d02f68@1959 "...
      arg$1 = 3
      arg$2 = 5
    ✓ S2 = ConjuntosDifusos$$Lambda$89/0x0000000800d02f68@1960 "t...
      arg$1 = 2
      arg$2 = 5
    > this = ConjuntosDifusos@1958
```

Tal como todas las funciones hasta ahora lo que hace esta función es llamar a “pertenece” pues necesita el valor de pertenencia de los números entre los intervalos para evaluar si pertenecen, el valor de pertenencia de S1 no puede ser mayor que los de S2. Pues estamos averiguando si puede pertenecer en dicho conjunto por lo que no puede tener más elementos que S2

```
✓ VARIABLES
  ✓ Local
    elem = 0
  ✓ s = ConjuntosDifusos$$Lambda$89/0x0000000800d02f68@1959 "t...
    arg$1 = 3
    arg$2 = 5
  > this = ConjuntosDifusos@1958
```

El paso siguiente es evaluarlo en S2 para observar lo anteriormente mencionado

```
✓ VARIABLES 
  ✓ Local
    elem = 0
  ✓ s = ConjuntosDifusos$$Lambda$89/0x0000000800d02f68@1960 "ta...
    arg$1 = 2
    arg$2 = 5
  > this = ConjuntosDifusos@1958
```

Cada vez que el programa puede seguir operando sin problemas este se detiene justamente en el limite que hemos puesto, ya que esta condición se cumple nos retorna que el conjunto “S1” si puede pertenecer a “S2”

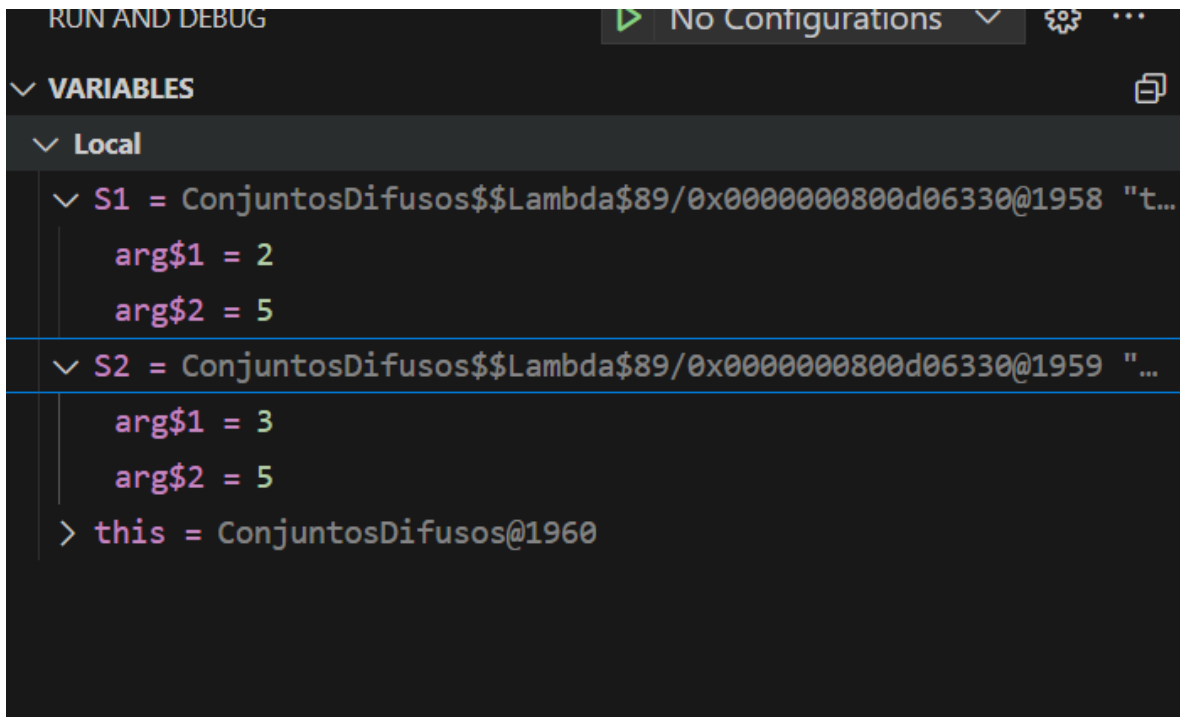
```
✓ VARIABLES
  ✓ Local
    elem = 10
  ✓ s = ConjuntosDifusos$$Lambda$89/0x0000000800d02f68@1960 "ta...
    arg$1 = 2
    arg$2 = 5
  > this = ConjuntosDifusos@1958

  > WATCH
```

INCLUSIÓN:

Se implemento la Función Inclusión que recibe dos conjuntos difusos para luego verificar si ambos conjuntos son iguales y por lo que para realizar esta función usamos: Pertenece e Inclusión.

Tal como se menciona antes esta función usa otras dos funciones para realizar lo indicado, por lo que realiza los llamados necesarios para ver si el funcionamiento es correcto, vamos a utilizar los mismos conjuntos de antes el de 2 y 5 junto al conjunto de 3 y 5



The screenshot shows a debugger interface with a dark theme. At the top, there's a 'RUN AND DEBUG' button and a 'No Configurations' dropdown. Below this is the 'VARIABLES' panel, which is expanded to show 'Local' variables. Two variables are listed: 'S1' and 'S2'. Both are of type 'ConjuntosDifusos' and have long hexadecimal memory addresses. 'S1' has arguments 'arg\$1 = 2' and 'arg\$2 = 5'. 'S2' has arguments 'arg\$1 = 3' and 'arg\$2 = 5'. Below these, there's a line '> this = ConjuntosDifusos@1960'.

```
RUN AND DEBUG No Configurations
VARIABLES
  Local
    S1 = ConjuntosDifusos$$Lambda$89/0x0000000800d06330@1958 "t...
      arg$1 = 2
      arg$2 = 5
    S2 = ConjuntosDifusos$$Lambda$89/0x0000000800d06330@1959 "...
      arg$1 = 3
      arg$2 = 5
    > this = ConjuntosDifusos@1960
```

Seguimos usando el mismo intervalo entre 0 y 10 para poder verificarlo, por lo que el ejercicio empieza a denuevo desde este punto

```
✓ VARIABLES   
  ✓ Local  
    elem = 0  
    ✓ s = ConjuntosDifusos$$Lambda$89/0x0000000800d06330@1958 "t...  
      arg$1 = 2  
      arg$2 = 5  
    > this = ConjuntosDifusos@1960
```

Tal como antes en cada ciclo el numero del va aumentando por cada iteración y cambiando el conjunto al que le aplica.

```
✓ VARIABLES   
  ✓ Local  
    elem = 1  
    ✓ s = ConjuntosDifusos$$Lambda$89/0x0000000800d06330@1959 "ta...  
      arg$1 = 3  
      arg$2 = 5  
    > this = ConjuntosDifusos@1960
```

Esta vez el programa debe de terminar aquí pues ha detectado que el conjunto 2 y 5 no puede estar dentro del conjunto 3 y 5, esto retorna un false el cual se aplicará al && de la función evitando que siga pues con un solo false en un AND basta para que todo sea False

```
✓ VARIABLES
  ✓ Local
    elem = 3
  ✓ s = ConjuntosDifusos$$Lambda$89/0x0000000800d02f68@1959 "ta...
    arg$1 = 3
    arg$2 = 5
  > this = ConjuntosDifusos@1960
```

2 INFORME DE CORRECCIÓN

Esto es realizado tomando en cuenta lo explicado en el taller usaremos lo otorgado por “pertenece” y “mucho mayor que”, además de obviamente lo explicado por “ConjDifusos” junto a su respectiva explicación

GRANDE

Se implemento la Función Grande que recibe dos números “d” el cual es un número mayor o igual a 1, luego recibe “e” el cual es un numero entero mayor a 1. Estos números son utilizados para calcular el grado de pertenencia de “d” en un conjunto difuso. Sea Grande el siguiente programa en Scala:

```
def grande(d: Int, e: Int): conjDifuso = n =  
> Math.pow(n.toDouble / (n + d), e)
```

En el caso base o sea $n = 0$ vamos a comprobar que 0 no es un numero grande al retornarnos un numero que no queda dentro de los grandes.

$$f_{grande}(0) = \left(\frac{0}{d + 0}\right)^e = 0.$$

Como se puede observar al tener un numero que es igual a 0 nos debe de retornar siempre un valor de 0.

Caso $n = (k + 1)$ en el que vamos a demostrar que la formula va a retornar un valor que siempre va en aumento con respecto a n .

$$f_{grande}(k + 1) = \left(\frac{(k + 1)}{d + (k + 1)}\right)^e = \left(\frac{k + 1}{d + k + 1}\right)^e$$

Lo comparamos con

$$f_{grande}(k) = \left(\frac{k}{d + k}\right)^e$$

Para ver si es mayor luego de igualarlos al multiplicarlos por sus denominadores

$$(k + 1)(k + d) > k(k + 1 + d)$$

Posteriormente solucionamos los paréntesis

$$k + d + k^2 + kd > k^2 + kd + k$$

Lo cual se simplifica a

$$d > 0$$

Un numero que siempre va a ser mayor a 0 mientras más grande sea la n

COMPLEMENTO

Se implemento la Función Complemento que recibe un “c” conjunto difuso para luego calcular cual es el complemento de un conjunto difuso, para eso debe de retornar un valor de pertenencia que tiene que ser distinto al de la pertenencia normal. Sea complemento el siguiente programa en Scala:

$$\begin{aligned} \text{def complemento}(c: \text{ConjDifuso}): \text{ConjDifuso} = n = \\ > 1.0 - \text{pertenece}(n, c) \end{aligned}$$

Para poder verificarlo hacemos lo siguiente: Primero usamos un caso en el que “n” pertenece al conjunto difuso y por ende su grado de pertenencia es 1 por lo que:

$$f(n) = 1.0 - 1.0 = 0.0$$

Lo cual obviamente esta correcto, pues algo que pertenece al conjunto no puede ser el complemento, por otro lado, revisamos con un “n” que no pertenece al conjunto

$$f(n) = 1.0 - 0.0 = 1.0$$

Como se puede notar funciona de la manera correcta al devolver un valor de pertenencia correcto a un valor fuera del conjunto.

Para lo siguiente debemos de asumir un segundo conjunto el cual tiene $k+1$ elementos y todos hasta k están correctos y posteriormente consideramos un elemento n_{k+1} , pero como se debe de aplicar la función “pertenece” y esta sigue devolviendo valores entre $[0,1]$. La función se vería así:

$$f(k + 1) = 1.0 - fpertenece(k + 1, c') = 1.0$$

UNIÓN

Se implemento la Función Complemento que recibe dos conjuntos difusos para luego calcular cual es el grado de pertenencia de un elemento en la unión de ambos, para eso debe de retornar un valor de pertenencia. Sea complemento el siguiente programa en Scala:

def union(cd1: ConjDifuso, cd2: ConjDifuso): ConjDifuso = n =>
Math.max(pertenece(n, cd1), pertenece(n, cd2))

- Si un elemento n pertenece a ambos conjuntos, el resultado será 1 (grado de pertenencia máximo).
- Si n pertenece solo a uno de los conjuntos, el grado de pertenencia será igual al de ese conjunto.
- Si n no pertenece a ninguno de los conjuntos, el resultado será 0.

Por lo que revisaremos varios casos

El primero será uno que pertenece al conjunto 1, pero no al conjunto 2

$$pertenece(n, cd1) = 1.0, pertenece(n, cd2) = 0.0$$

Por lo que n debe de pertenecer a la unión tal que:

$$f(n) = \max(1.0, 0.0) = 1.0$$

Esto funciona de igual manera al revés gracias a que se saca el valor máximo por lo que:

$$pertenece(n, cd1) = 0.0, pertenece(n, cd2) = 1.0$$

$$f(n) = \max(0.0, 1.0) = 1.0$$

Posteriormente siguen los casos donde pertenecen ambos y lo que retorna

$$pertenece(n, cd1) = 0.3, pertenece(n, cd2) = 0.4$$

$$f(n) = \max(0.3, 0.4) = 0.4$$

Y finalizamos con el que no pertenece ninguno

$$pertenece(n, cd1) = 0.0, pertenece(n, cd2) = 0.0$$

$$f(n) = \max(0.0, 0.0) = 0.0$$

Tal como con complemento en este caso el caso de inducción se simplifica ya que la función esta usando a pertenece consigo, por lo que los resultados se mantendrán entre [0,1] por lo que incluso así:

$$f(n_{k+1}) = \max(pertenece(n_{k+1}, cd1), pertenece(n_{k+1}, cd2))$$

Los valores que nos devolverá estarán entre [0,1] manteniendo la propiedad de la función al devolvernos el grado de pertenencia correcto.

INTERSECCIÓN

Se implemento la Función Complemento que recibe dos conjuntos difusos para luego calcular cual es el grado de pertenencia de un elemento en la unión de ambos, para eso debe de retornar otro un valor de pertenencia. Sea complemento el siguiente programa en Scala:

def interseccion(cd1: ConjDifuso, cd2: ConjDifuso): ConjDifuso = n =>

Math.min(pertenece(n, cd1), pertenece(n, cd2))

- Si un elemento **n** pertenece a ambos conjuntos, el resultado será el menor grado de pertenencia entre ambos.
- Si **n** pertenece a solo uno de los conjuntos, el resultado será el grado de pertenencia de ese conjunto.
- Si **n** no pertenece a ninguno de los conjuntos, el resultado será 0.

El primero será uno que pertenece al conjunto 1, pero no al conjunto 2

$$pertenece(n, cd1) = 1.0, pertenece(n, cd2) = 0.0$$

Por lo que n debe de pertenecer a la unión tal que:

$$f(n) = \min(1.0, 0.0) = 0.0$$

Esto funciona de igual manera al revés gracias a que se saca el valor máximo por lo que:

$$pertenece(n, cd1) = 0.0, pertenece(n, cd2) = 1.0$$

$$f(n) = \min(0.0, 1.0) = 0.0$$

Posteriormente siguen los casos donde pertenecen ambos y lo que retorna

$$pertenece(n, cd1) = 0.3, pertenece(n, cd2) = 0.4$$

$$f(n) = \min(0.3, 0.4) = 0.3$$

Y finalizamos con el que no pertenece ninguno

$$pertenece(n, cd1) = 0.0, pertenece(n, cd2) = 0.0$$

$$f(n) = \min(0.0, 0.0) = 0.0$$

Tal como con complemento en este caso el caso de inducción se simplifica ya que la función está usando a “pertenece” consigo, por lo que los resultados se mantendrán entre $[0,1]$ por lo que incluso así:

$$f(n_{k+1}) = \min(pertenece(n_{k+1}, cd1), pertenece(n_{k+1}, cd2))$$

Los valores que nos devolverá estarán entre $[0,1]$ manteniendo la propiedad de la función al devolvernos el grado de pertenecía correcto.

INCLUSIÓN

Se implemento la Función Inclusión que recibe dos conjuntos difusos para luego verificar si un conjunto difuso puede pertenecer dentro de otro conjunto difuso, por lo que para eso usaremos recursión. Sea Intersección el siguiente programa en Scala:

```
def inclusion(S1: ConjDifuso, S2: ConjDifuso): Boolean
= { @annotation.tailrec

    def aux(n: Int, max: Int): Boolean = {
        if (n > max) true
        else if (pertenece(n, S1) > pertenece(n, S2)) false
        else aux(n + 1, max)
    }
    aux(0, 1000)
}
```

Para realizar esta función se hace lo siguiente:

- Al hacer la operación debe de retornar un booleano, por ejemplo, si c1 está incluido en c2 debe de retornar True, pero en caso de que no debe de regresar False.
- al evaluar todos los casos si encuentra uno que sea cuya pertenencia sea $c1 < c2$ deberá retornar False en caso contrario retornará True
- Evaluar el grado de pertenencia entre $[1, 1000]$ esto debido a que la función pertenece evalúa un numero en un conjunto.

$$pertenecer(n, c1) = < pertenecer(n, c2)$$

No hay nada en c1 y c2

En este caso todos los conjuntos están vacíos entre n y c1 o c2, por lo que ambos retornarían lo siguiente:

$$pertenecer(n, c1) = 0, pertenecer(n, c2) = 0$$

Por lo que lo que retornaría sería un True al seguir evaluando bajo la lógica impuesta.

$$c1 > 0 \text{ y } c2 = 0$$

En este caso lo que evaluará la función será lo siguiente:

$$pertenecer(n, c1) > 0, pertenecer(n, c2) = 0$$

En el que retornará False al cumplirse la condición de que c1 no puede ser mayor que c2.

Ambos conjuntos son desiguales

En este caso lo que pasará es que al ser el c1 mayor que el c2 retornará false, pero en el caso de que c1 sea menor a c2 este deberá retornar True, tal que:

$$\text{Inclusión}(pertenecer(n, c1) > pertenecer(n, c2)) = \text{False}$$

$$\text{Inclusión}(pertenecer(n, c1) < pertenecer(n, c2)) = \text{True}$$

Ambos conjuntos son iguales en este caso ambos conjuntos son iguales, por lo que retornaría es lo siguiente, cumpliendo las condiciones:

$$\text{Inclusión}(pertenecer(n, c1) < pertenecer(n, c2)) = \text{True}$$

IGUALDAD

Se implemento la Función Inclusión que recibe dos conjuntos difusos para luego verificar si ambos conjuntos son iguales y por lo que para realizar esta función usamos: Pertenece e Inclusión. Sea Igualdad el siguiente programa en Scala:

```
def igualdad(S1: ConjDifuso, S2: ConjDifuso): Boolean =  
    inclusion(S1, S2) && inclusion(S2, S1)
```

Para realizar esta función se hace lo siguiente:

- Al hacer la operación debe de retornar un booleano

-al evaluar todos los casos si encuentra uno que sea cuya pertenencia sea $c1 < c2$ o $c2 < c1$ deberá retornar False en caso contrario retornará True

-Evaluar el grado de pertenencia entre $[1, 1000]$ esto debido a que la función pertenece evalúa un numero en un conjunto.

$$pertenece(n, c1) > pertenece(n, c2)$$

No hay nada en $c1$ y $c2$

En este caso todos los conjuntos están vacíos entre n y $c1$ o $c2$, por lo que ambos retornarían lo siguiente:

$$\begin{aligned} & pertenece(n, c1) = 0, pertenece(n, c2) = 0 \\ & (pertenece(n, c1) = 0 \text{ } pertenece(n, c2)) = 0 \ \&\& \ (pertenece(n, c1) \\ & \quad = 0 \text{ } pertenece(n, c2)) = True \end{aligned}$$

Por lo que lo que retornaría sería un True al seguir evaluando bajo la lógica impuesta.

$C1 > 0$ y $c2 = 0$

En este caso lo que evaluará la función será lo siguiente:

$$\begin{aligned} & pertenece(n, c1) > 0, pertenece(n, c2) = 0 \\ & (((pertenece(n, c1) > pertenece(n, c2)) = False \ \&\& \ (pertenece(n, c2) \\ & \quad < pertenece(n, c1)) = True) = True \end{aligned}$$

En el que retornará False al cumplirse la condición de que $c1$ no puede ser mayor que $c2$.

$C1 = 0$ y $c2 > 0$

En este caso lo que evaluará la función será lo siguiente:

$$\begin{aligned} & pertenece(n, c1) = 0, pertenece(n, c2) > 0 \\ & (((pertenece(n, c1) < pertenece(n, c2)) = True \ \&\& \ (pertenece(n, c2) \\ & \quad > pertenece(n, c1)) = False) = True \end{aligned}$$

En el que retornará False al cumplirse la condición de que $c1$ no puede ser mayor que $c1$.

Ambos conjuntos son desiguales

En este caso lo que pasará es que el al ser el $c1$ mayor que el $c2$ retornará false, pero en el caso de que $c1$ sea menor a $c2$ este deberá retornar True, tal que:

$$pertenece(n, c1) > pertenece(n, c2)$$

$$(((pertenece(n, c1) > pertenece(n, c2)) = False \ \&\& \ (pertenece(n, c2) < pertenece(n, c1)) = True) = True$$

En el que retornará False al cumplirse la condición de que c1 no puede ser mayor que c2.

Ambos conjuntos son iguales

Ambos conjuntos son iguales en este caso ambos conjuntos son iguales, por lo que retornaría es lo siguiente, cumpliendo las condiciones:

$$pertenece(n, c1) = pertenece(n, c2)$$

$$(((pertenece(n, c1) = pertenece(n, c2)) = True \ \&\& \ (pertenece(n, c2) = pertenece(n, c1)) = True) = True$$